Article Recommendation System

Collaborative Filtering using LSH and Content-Based Filtering with Clustering

Name: Antoine Abdallah
Email: antoine.abdallah@studenti.unimi.it
Matriculation Number: 79051A

Course: Algorithms for Massive Datasets
Professor: Dario Malchiodi
Academic Year: 2025–2026

March 20, 2026

Github repository

1 Summary

The objective of this project is to implement a scalable recommendation engine for New York Times articles. The system identifies latent relationships between users based on their commenting history. By employing **Locality Sensitive Hashing (LSH)**, the system maps users into discrete buckets to find "nearest neighbors" without the quadratic complexity of an all-pairs comparison. To refine these results, a hybrid approach is used: collaborative filtering to identify potential articles, while a content-based clustering layer ensures that recommendations align with the thematic interests of the target user.

2 The Process Flow

The recommendation logic follows a specific pipeline designed to handle high-dimensional user-item interactions:

1. **Identification:** When User A interacts with an article, the system identifies other users (e.g., User B) who shared that interaction.
2. **Candidate Generation:** Using LSH, the system retrieves a set of candidate users who have similar overall commenting profiles to User A.
3. **Article Retrieval:** The system fetches all articles User B has interacted with that User A has not yet seen.
4. **Thematic Filtering:** These candidate articles are filtered against a pre-computed clustering model to ensure they match User A's preferred content categories.

3 Phase 1: Article Clustering and Dimensionality Reduction

The first phase involves organizing the article corpus into thematic groups using metadata.

3.1 Categorical Preprocessing and One-Hot Encoding

Articles are defined by: *newsdesk*, *section*, *subsection*, and *material*. These represent categorical variables. We apply **One-Hot Encoding (OHE)** to transform these into a high-dimensional binary vector space. This ensures that the distance between "Sports" and "Politics" is equidistant, preventing the model from assuming any numerical relationship between text labels.

3.2 Truncated SVD (Latent Semantic Analysis)

Because OHE creates a highly sparse matrix where most entries are zero, we apply **Truncated SVD**.

- **Functionality:** Truncated SVD performs a factorization of the sparse matrix to project it into a lower-dimensional "latent" space.

- **Application:** This reduces noise and captures the underlying "concepts" of the articles. By compressing the thousands of OHE dimensions into a dense set of components, we make the subsequent clustering computationally feasible while preserving the most significant variance in the data.

3.3 K-Means Clustering and Optimization

Using the reduced SVD components, we apply the K-Means algorithm. To ensure the clusters are meaningful, we use the **Elbow Method** to find the point where increasing k no longer significantly improves the Within-Cluster Sum of Squares (WCSS).

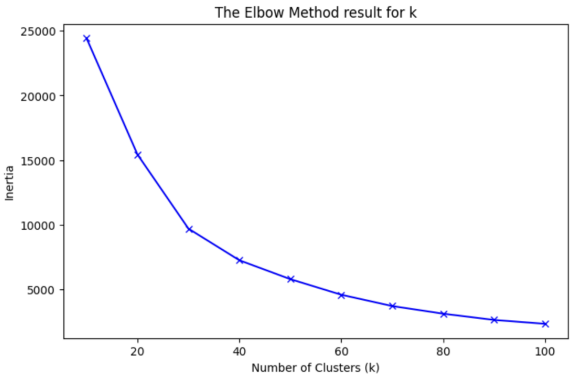

Figure 1: Elbow Method Analysis ($k = 25$)

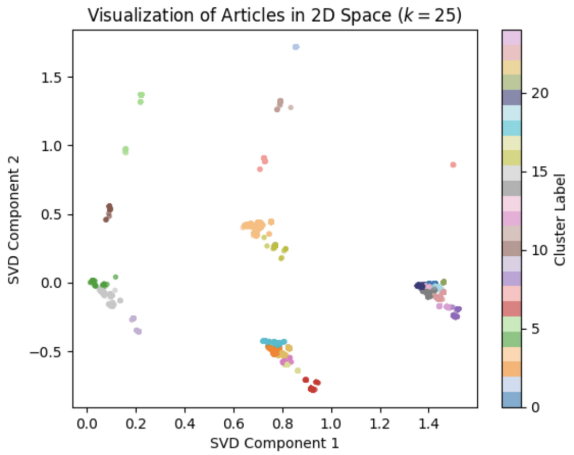

Figure 2: Article Cluster Distribution

4 Phase 2: LSH with Baseline Hashing

In Phase 2, we establish the LSH framework using a **hard-coded hash function**. The primary goal here is to implement the MinHash logic: converting the sparse user-article interaction matrix into a signature matrix. This baseline allows us to verify the correctness of the banding technique and the retrieval of candidate pairs based on Jaccard similarity approximations.

5 Phase 3: Optimized LSH with mmh3

Phase 3 replaces the baseline hash with the **MurmurHash3** library to achieve production-grade performance.

- **Functionality of `mmh3_x64_128_digest()`:** This specific function is utilized to generate 128-bit hashes. Unlike cryptographic hashes (like MD5), it is designed for high-speed data processing.
- **Optimization:** The `x64` variant is optimized for 64-bit CPU architectures, allowing it to process 16 bytes of data per cycle. By using the `digest()` output, we obtain a raw byte representation that minimizes memory overhead when storing millions of bucket keys.

- **Distribution:** The 128-bit width provides a massive hash space (2^{128}), virtually eliminating accidental collisions between unrelated users and ensuring the LSH buckets are highly precise.

6 Recommendation Functionalities

The final system relies on two distinct retrieval functions. To maintain high execution speeds during the evaluation of massive datasets, only 50% of the available sparse matrix was processed.

6.1 `find_recommendations`

This function acts as the primary collaborative filter. It queries the LSH structure to find users who "land" in the same hash buckets as the target user. It then performs a set difference operation:

$$\text{Recs} = \{\text{Articles of User B}\} \setminus \{\text{Articles of User A}\}$$

This ensures the recommendations consist strictly of new content.

6.2 `find_clustered_recommendations`

This function integrates the content-based clustering from Phase 1. It first identifies the specific cluster IDs associated with User A's history. It then executes `find_recommendations` but applies a secondary filter: only articles belonging to the user's identified "interest clusters" are returned. This hybrid approach filters out "random" articles that User B might have read, focusing strictly on relevant thematic content.